

Projet Logiciel Transversal: SmallWorld

Julien METZELARD – Tarek TALSI – Maël ARCHENAUULT – Victor MOREL

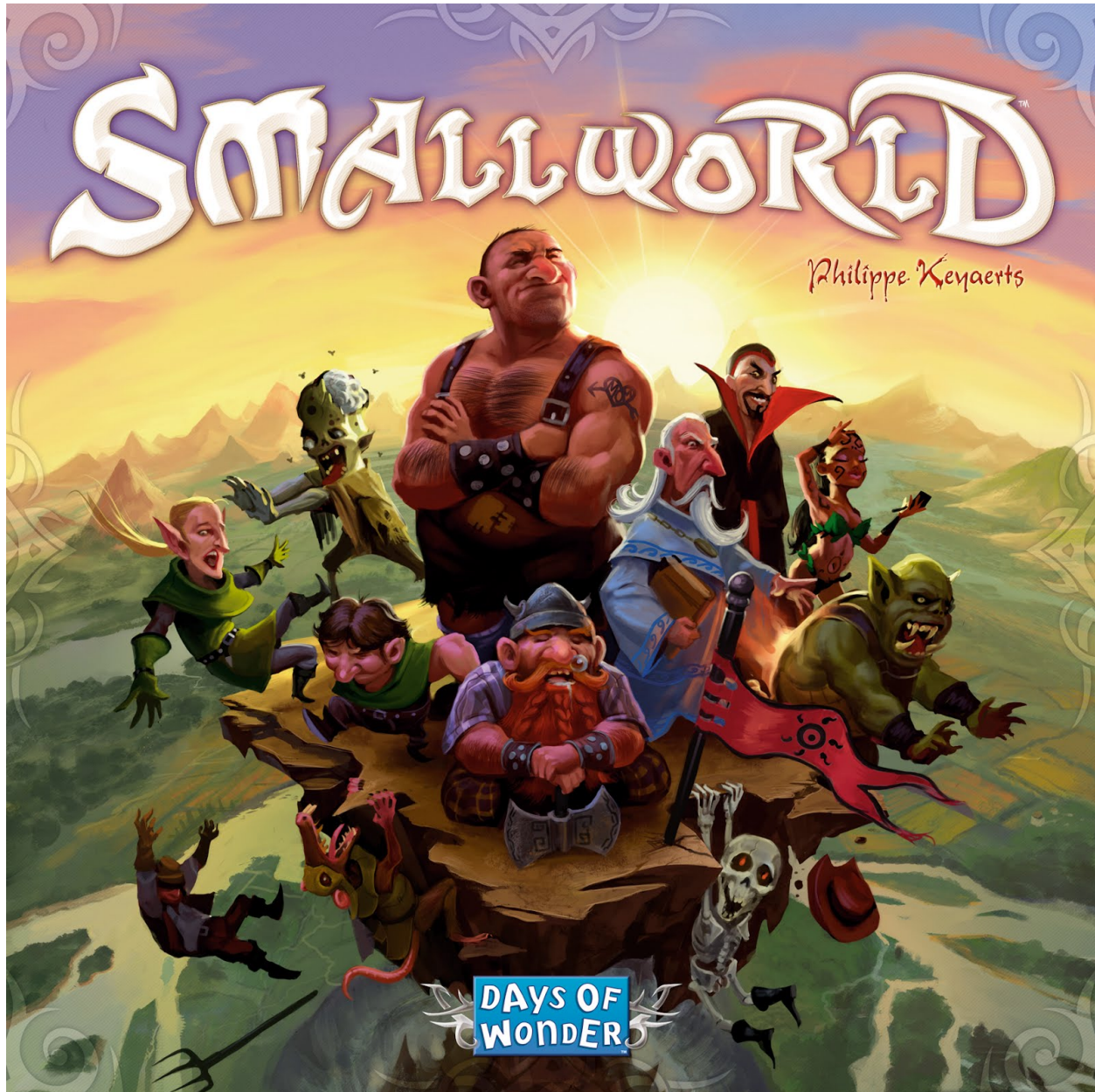


Figure 1: Illustration sur la boîte du jeu

1 Objectif

1.1 Présentation générale

L'objectif de ce projet est de créer une version numérique du jeu de plateau "SmallWorld". Le projet est mené par 4 étudiants. Le jeu est codé en C++.

1.2 Règles du jeu

Le jeu choisi est un jeu de stratégie en tour par tour. Pour gagner, les joueurs doivent avoir à la fin du jeu le plus d'argent.

SmallWorld se joue grâce à un plateau représentant une carte de territoires, et des pions faisant office de troupes. Lorsque son tour arrive, le joueur doit attaquer les territoires de ses adversaires afin de conquérir un maximum de terrain. A la fin de son tour, il reçoit autant d'argent que de terrains qu'il possède.



Figure 2: Plateau de jeu SmallWorld

À ces mécaniques de bases s'ajoutent des systèmes de pouvoirs. Chaque joueur choisit une "espèce" pour ses troupes ainsi qu'un "pouvoir". Les combinaisons espèce/pouvoir sont définies de manière aléatoire. Chaque espèce a un effet différent, de même pour les pouvoirs. Cela peut aller d'un bonus de troupes lors d'une attaque à un bonus de récompense à la fin du tour. Les pouvoirs sont très variés.

Chaque joueur peut, s'il le veut, abandonner son espèce actuelle et en prendre une nouvelle. L'espèce abandonnée reste sur le plateau, mais est marquée comme "en déclin". Les territoires occupés par une espèce en déclin rapportent toujours des récompenses à son ancien propriétaire.

Tout l'intérêt du jeu réside dans la capacité à changer d'espèce au bon moment, et à choisir la bonne combinaison espèce/pouvoir parmi celles proposées.

1.3 Conception Logiciel

Présenter ici les packages de votre solution, ainsi que leurs dépendances.

2 Description et conception des états

2.1 Description des états

L'état global du jeu est centralisé dans la classe `Game_State`, qui contient toutes les informations nécessaires pour décrire la partie à un instant donné :

- la carte (`Map`) contenant l'ensemble des informations sur les zones de la carte avec ses spécificités ainsi que les troupes des joueurs,
- la liste des joueurs (`Player`),
- les tribus (combinaison d'espèce et de pouvoir) disponibles via une pile (`Tribe_Stack`),
- des paramètres de gestion de tour comme le nombre de joueur actif ainsi que le nombre de rounds.

Chaque `Player` représente un joueur de la partie et possède :

- un identifiant unique,
- un ensemble de tribus actives qu'il contrôle,
- un compteur de points de victoire accumulés,
- les méthodes associées à la conquête ou au déploiement d'unités.

Une `Tribe` correspond à la combinaison d'une espèce et d'un pouvoir spécial, ce qui définit les capacités et bonus de la tribu. Chaque tribu possède :

- un nombre d'unités disponibles,
- des descriptions (`Species_Description`, `Power_Description`) déterminant ses effets,
- des méthodes associées aux tribus comme `go_in_decline` et autre qui seront explicités plus tard.

Les zones (`Area`) modélisent les régions de la carte. Elles contiennent :

- un biome (`Area_Biome`) et éventuellement des spécial tokens (forteresse, tanière, etc.),
- une liste de voisins, pour représenter les connexions de la carte,
- un propriétaire (`Tribe`), ici il est important que le propriétaire soit une `Tribe` et non pas un `Player` car un `Player` peut avoir plusieurs `Tribe` et que ces `Tribe` peuvent s'attaquer mutuellement,
- et des méthodes permettant la conquête ou le déploiement d'unités.

L'ensemble de ces zones est géré par la classe `Map`, qui stocke leur liste et permet le chargement depuis un fichier JSON (utilisé pour initialiser la carte).

Enfin, les effets spéciaux sont gérés par la hiérarchie `Effects_Bundle` :

- La classe abstraite `Effects_Bundle` définit une interface générique (`apply_first_round_effect`, `apply_conquest_effect`, etc.).
- Des classes concrètes comme `Dwarf_Effects_Bundle`, `Ratmen_Effects_Bundle` ou `Giant_Effects_Bundle` héritent de cette interface pour appliquer des bonus spécifiques. Il est important de préciser que le diagramme UML actuel ne contient pas toutes les classes concrètes car chacune d'entre-elles "viole" les règles du jeu ce qui complique leur implémentation.

2.2 Conception logicielle

Le diagramme UML présente l'architecture des classes du module state. L'organisation suit une approche orientée objet modulaire (plus que cruciale dans notre cas avec autant d'effet différent) :

- `Game_State` joue le rôle de façade, offrant un point d'entrée unique pour manipuler l'état global du jeu.
- `Map` et `Area` forment le modèle spatial, décrivant la topologie/état physique du monde.
- `Player` et `Tribe` constituent le modèle des acteurs de ce monde, reliés entre eux par composition.
- `Species_Description`, `Power_Description` et `Effects_Bundle` réalisent le modèle des caractéristiques et utilisent une hiérarchie d'héritage pour encapsuler les comportements spécifiques à chaque race ou pouvoir.

3 Rendu: Stratégie et Conception

3.1 Stratégie de rendu d'un état

Le jeu possède différentes structures assez différentes à afficher: - **la carte de jeu** : Map - **les pions posés sur les territoires** : Units, Special_Tokens, etc ... - **la pioche d'espèce** : Tribe_Stack - **la main du joueur** : Money, Free_Units, Tribes

Pour afficher ces différentes structures, nous avons choisi de créer une classe `Renderer` qui va s'occuper de tout le rendu graphique du jeu. Cette classe va utiliser la bibliothèque SFML pour afficher les différentes structures du jeu.

Chaque structure va avoir une classe dédiée à son affichage. Cela permettra une meilleure organisation du code et une meilleure réutilisabilité.

3.2 Conception logicielle

Voici une description des classes principales utilisées pour le rendu:

Token_Renderer

Cette classe est responsable du rendu des tokens (unités, special tokens, area_specialization) sur la carte. C'est une brique de base utilisée dans d'autres classes de rendu (Map_Renderer et Player_Area_Renderer)

Avant de rendre l'objet sur une fenêtre de rendu, on peut changer ce qu'il représente:

- pour des pions: l'espèce sur le pion
- pour des special_tokens: le type de special_tokens
- pour des area_specialization: le type d'area_specialization

Il est aussi possible d'ajouter un nombre sur le côté de l'affichage du pion pour représenter le nombre d'unités présentes sur la zone.

Tribe_Renderer

Cette classe est responsable du rendu des tribes. Une tribu est affichée grâce à deux cartes:

- une carte de pouvoir
- une carte d'espèce

Ce moteur de rendu est capable de rendre l'ensemble des deux cartes (en les fixant l'une par rapport à l'autre).

Map_Renderer

Cette classe est responsable du rendu de la carte de jeu. Elle utilise la classe `Token_Renderer` pour afficher les unités et les special tokens sur chaque zone de la carte.

La carte de jeu est stockée sous la forme d'une image associée avec un fichier json représentant les positions des différentes zones. Le rendu de la carte se fait en deux étapes: - on affiche l'image de la carte - on affiche les unités et les special tokens sur chaque zone grâce à la classe `Token_Renderer`

Player_Area_Renderer

Cette classe est responsable du rendu de la main d'un joueur donné. Elle utilise la classe `Token_Renderer` pour afficher les unités que le joueur possède dans sa main. De même la classe `Tribe_Renderer` est utilisée pour afficher les tribus que le joueur possède.

La classe `Player_Area_Renderer` possède une référence vers la Map qu'elle affiche.

Trois choses sont affichées par ce moteur de rendu: - l'argent du joueur - les unités libres du joueur - les tribus que le joueur possède (dont celles en déclin)

Tribe Stack Renderer

Cette classe est responsable du rendu de la pioche d'espèces. Elle utilise la classe `Tribe_Renderer` pour afficher les tribus disponibles dans la pioche.

La classe `Tribe_Stack_Renderer` possède une référence vers la `Tribe_Stack` qu'elle affiche.

Ce moteur de rendu affiche les 6 tribus de la pioche auquel le joueur a accès. Il ajoute à leur côté le prix de retrait de la tribu.

Renderer

Cette classe est responsable du rendu global du jeu. Elle utilise les différentes classes de rendu pour afficher l'état actuel du jeu.

3.3 Ressources

L'ensemble des ressources est stockée sous forme de spritesheet. Les classe s'occupent ensuite de d'afficher seulement une partie rognée de la spritesheet. Pour cela, elles s'aident de fichier nommes "indexing" qui leur indiquent où trouver chaque sprite dans la spritesheet.



Figure 4: Exemple de spritesheet

Exemple de fichier de type "indexing"

```
{
  "sprite_size": [150, 150],
  "frames": {
    "Amazons": [0,0],
    "Dwarves": [0,1],
    "Elves": [0,2],
    "Ghouls": [0,3],
    "Ratmen": [0,4],
    "Skeletons": [0,5],
    "Sorcerers": [0,6],
    "Tritons": [0,7],
    "Giants": [1,0],
    "Halflings": [1,1],
    "Humans": [1,2],
    "Orcs": [1,3],
    "Trolls": [1,4],
    "Wizards": [1,5],
    "Lost Tribe": [1,6]
  }
}
```


3.4 Exemple de rendu

A l'aide de la classe `Render`, il est possible de rendre l'état complet du jeu.



Figure 5: Exemple de rendu complet

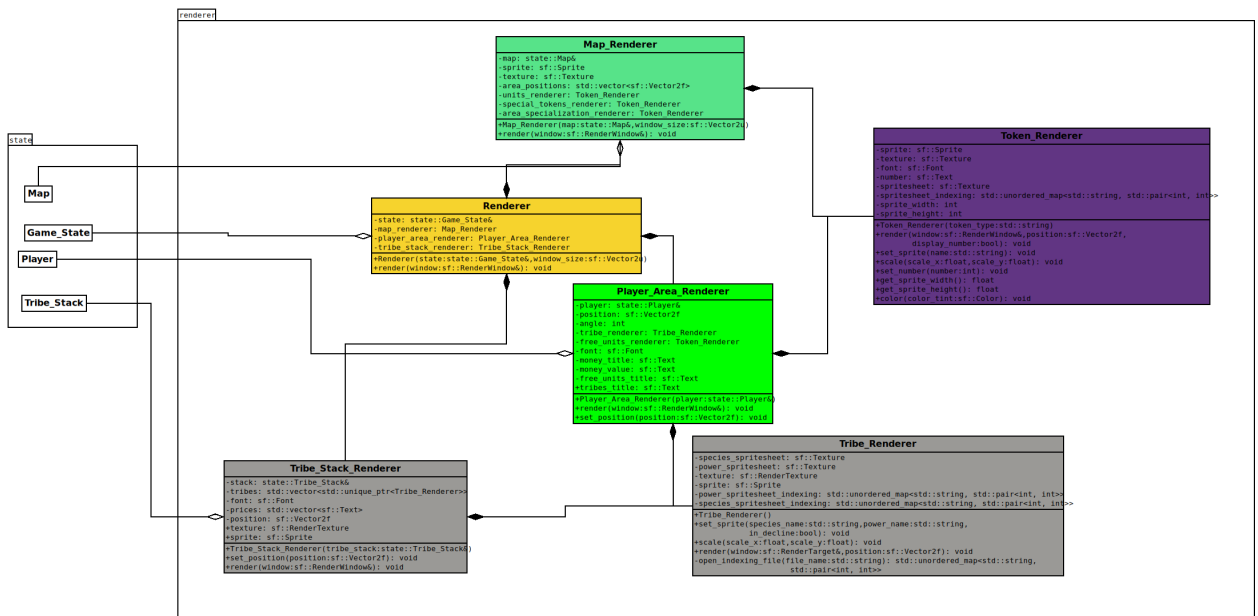


Figure 6: Diagramme de classes du module de rendu

4 Règles de changement d'états et moteur de jeu

Vue d'ensemble

Le moteur de jeu (« Engine ») est la couche responsable de l'exécution des actions des joueurs sur l'état de la partie (`state::Game_State`). Il ne contient pas la logique métier des règles (qui reste dans `Game_State`, `Area`, `Player`, etc.), mais orchestre l'application ordonnée des commandes produites par l'interface joueur, l'IA ou les tests.

Principales responsabilités : - recevoir et mettre en file (FIFO) des commandes via `add_command()` ; - valider la légitimité d'une commande par rapport à l'état courant (joueur courant, phase de tour, présence d'une tribu active, ...); - exécuter la commande en appelant `command->execute(state)` ; - gérer la file de commandes (suppression après exécution, possibilité d'annuler la commande en tête via `remove_last_command()` dans l'implémentation actuelle).

Cycle de traitement d'une commande

1. Appel de `add_command(std::unique_ptr<Command>)` pour mettre la commande dans la file.
2. Périodiquement (ou à chaque boucle de jeu), `Engine::update()` est appelé :
 - si la file est vide : rien à faire ;
 - sinon, le moteur prend la commande en tête et effectue des validations :
 - l'utilisateur possède-t-il une tribu active (sauf pour `Choose_Species`) ?
 - est-ce le tour du joueur qui a initié la commande ?
 - la commande est-elle autorisée dans la phase courante ?
 - en cas d'échec d'une de ces validations, la commande est retirée de la file et une exception `std::runtime_error` est lancée ;
 - si les validations passent, le moteur appelle `command->execute(state)` puis retire la commande de la file.

Règles de validation importantes

- `Choose_Species_Command` est spécial : il nécessite l'absence d'une tribu active pour le joueur.
- Les autres commandes (`Conquer`, `Redeploy`, `Decline`, ...) exigent la présence d'une tribu active.
- Le joueur qui envoie la commande doit être le joueur courant.
- La commande doit correspondre à la phase du tour en cours (ex. : `Conquer_Command` dans `CONQUER`).

Phases de tour

Le moteur de jeu s'appuie sur le concept de phases de tour (`state::Turn_Phase`) pour organiser les actions possibles à chaque étape du tour d'un joueur. Chaque commande est associée à une phase spécifique durant laquelle elle est autorisée. Les différentes phases sont les suivantes :

- `START` : l'utilisateur peut choisir une espèce, passer en déclin ou ne rien faire.
- `CONQUER` : l'utilisateur peut conquérir des territoires.
- `REDEPLOY` : l'utilisateur peut redéployer ses unités.

Les différentes commandes

Nom de la commande	Attributs (principaux)	Phase de tour d'usage	Phase suivante
<code>Choose_Species_Command</code>	<code>player_id</code> , <code>position</code>	<code>START</code>	<code>CONQUER</code>
<code>Decline_Command</code>	<code>player_id</code>	<code>START</code>	<code>START</code> (passe au joueur suivant)
<code>Start_Conquest_Command</code>	<code>player_id</code>	<code>START</code>	<code>CONQUER</code>
<code>Conquer_Command</code>	<code>player_id</code> , <code>attacked_area_id</code> , <code>n_units</code> , <code>dice_units</code>	<code>CONQUER</code>	<code>REDEPLOY</code> si dernière conquête, sinon reste en <code>CONQUER</code>

Nom de la commande	Attributs (principaux)	Phase de tour d'usage	Phase suivante
End_Conquer_Command	player_id	CONQUER	REDEPLOY
Redeploy_Command	player_id, area_id, added_units	REDEPLOY	START si plus d'unités libres (fin du tour) et passe au joueur suivant, sinon reste en REDEPLOY

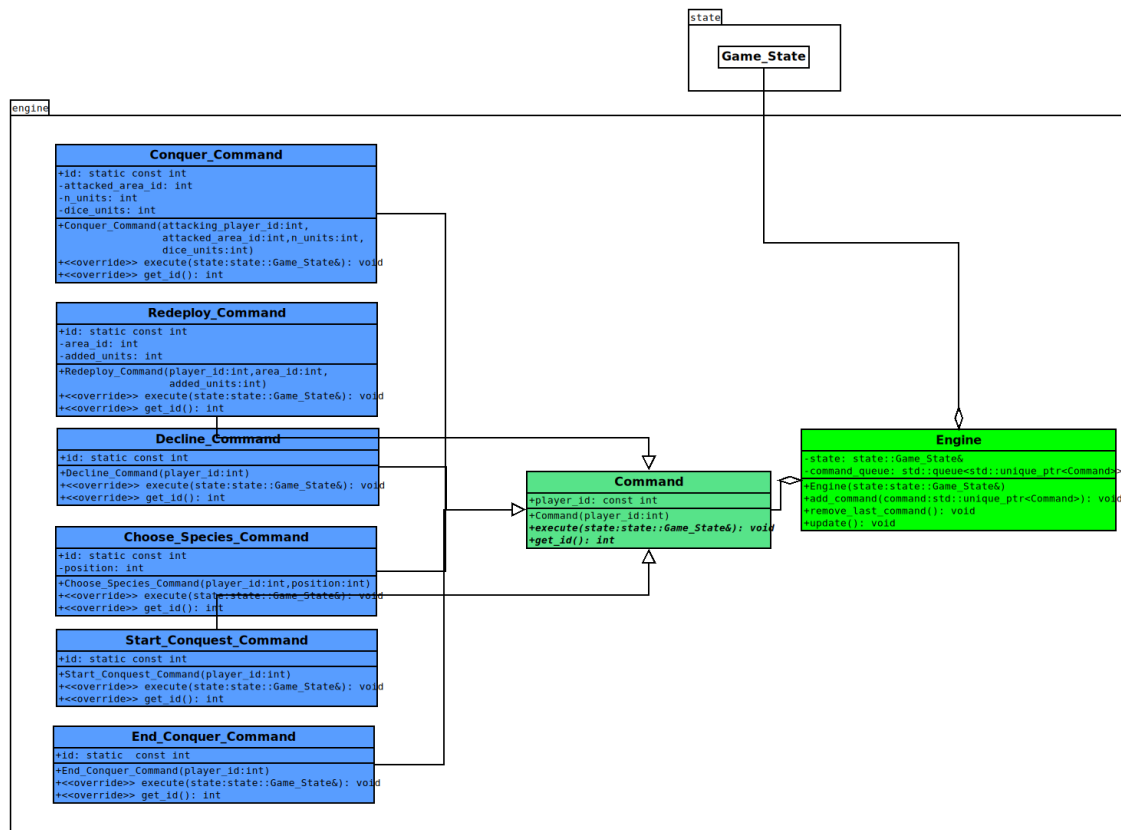


Figure 7: Diagramme de classes du moteur de jeu